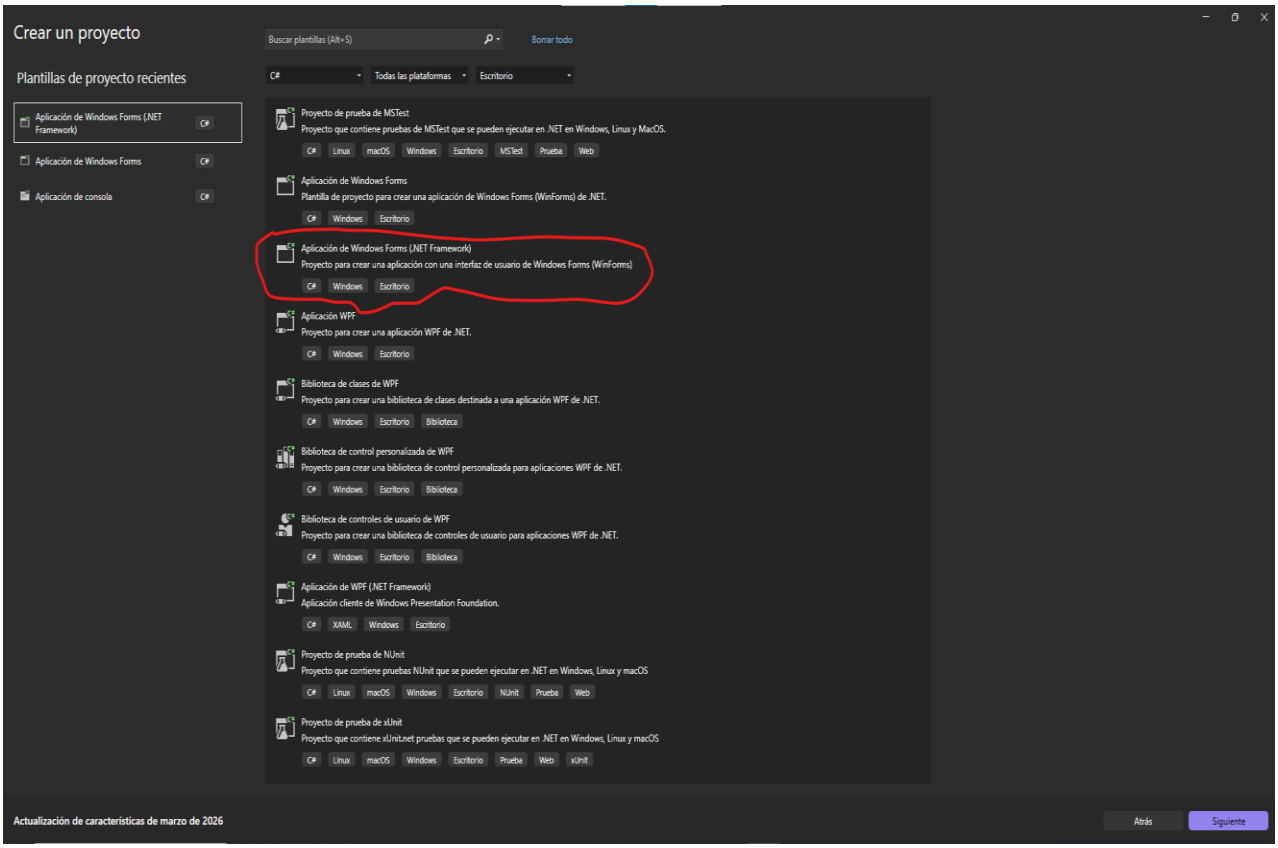
Guía Práctica II: Procesamiento paralelo, concurrencia y sincronía

Ejercicio práctico en Windows Forms con patrón Repository

1. Abrir Visual Studio 2022.
2. Crear nuevo proyecto Windows Forms App (.NET).
3. Asegúrese de que el Framework seleccionado sea .NET 6.0 o superior.

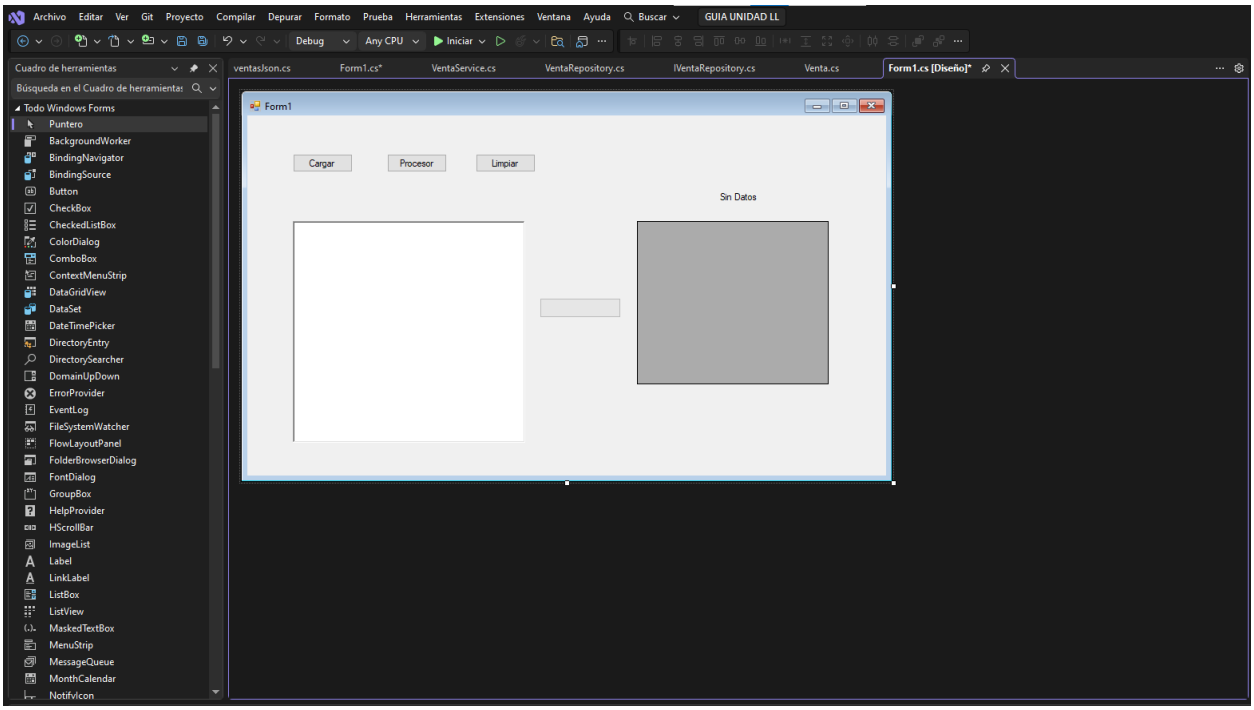


4.En la solución, agregue una nueva interfaz en este caso se llamará Ventas para una tienda

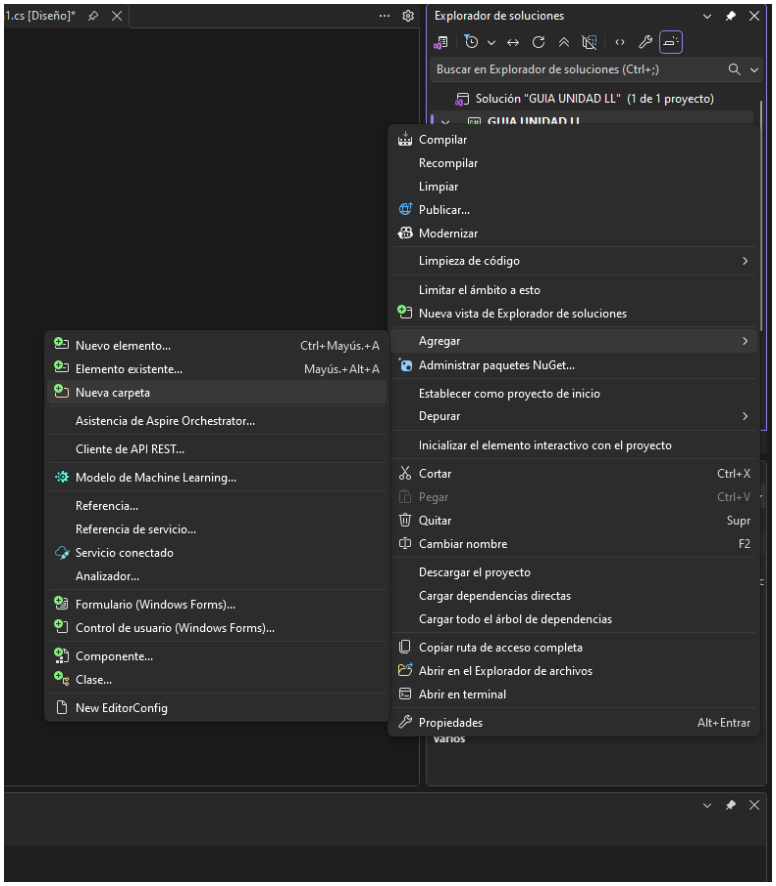
5. Crear una interfaz Grafica que lleve

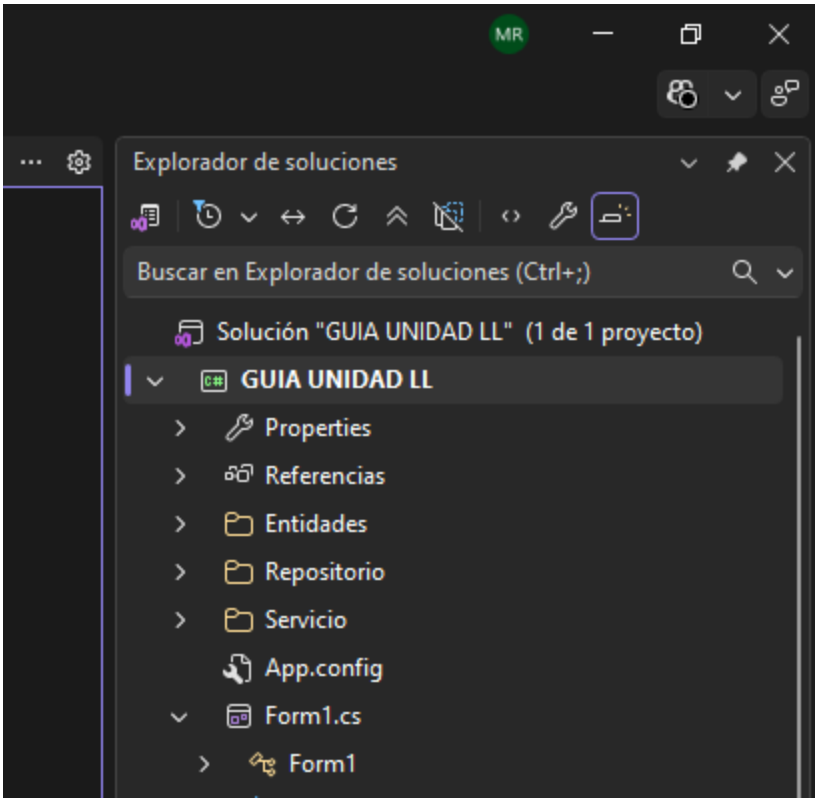
- **DataGridView**
  Nombre: dgvVentas
- **Button**
  Nombre: btnCargar
  Texto: Cargar ventas
- **Button**
  Nombre: btnProcesar
  Texto: Procesar ventas
- **Button**
  Nombre: btnLimpiar
  Texto: Limpiar
- **Label**
  Nombre: lblEstado
  Texto inicial: Sin datos
- **ProgressBar**
  Nombre: progressBar1
- **RichTextBox o TextBox multilínea**
  Nombre: txtResultado

Que se vea de la siguiente Manera



6. Luego en la parte de Explorador de soluciones, se le da Click derecho al Proyecto y se Crean 3 Carpetas con los nombres de Entidades, Repositorio, Servicio.





7. Luego en las Carpetas se Agregan Clases y una interfaz. Las clases y interfaz se crean dándole Click derecho a cada una de las carpetas y se agrega nuevo elemento y Interfaz.

Entidades

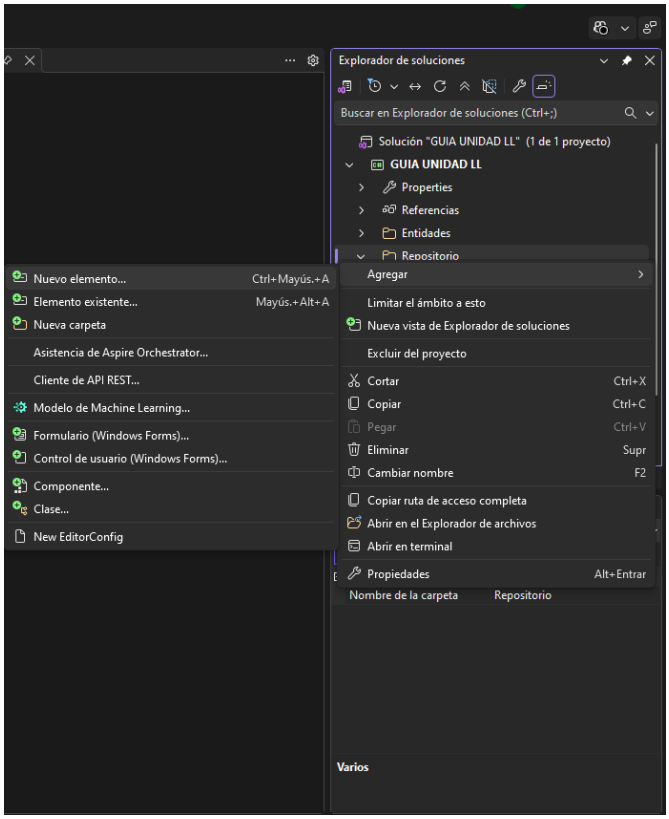
- **Venta.cs**

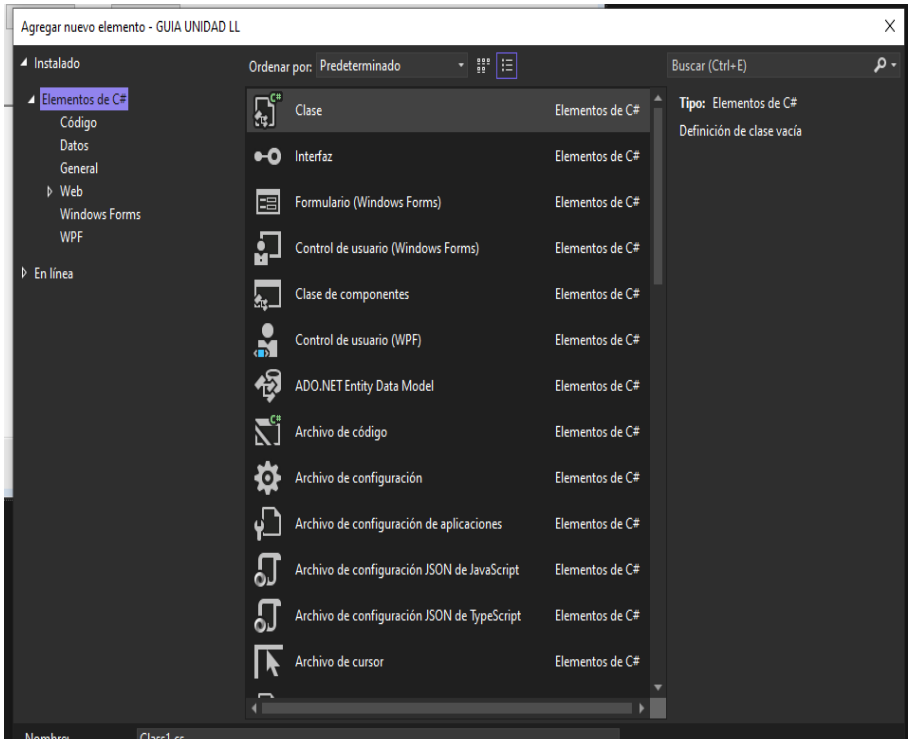
Repositorios

- **IVentaRepository.cs**
- **VentaRepository.cs**

Servicios

- **VentaService.cs**





8. En el archivo Venta.cs copíar el siguiente código:

```
public class Venta
 {
    public int Id { get; set; }
    public string Producto { get; set; }
    public string Categoria { get; set; }
    public int Cantidad { get; set; }
    public decimal PrecioUnitario { get; set; }
    public DateTime Fecha { get; set; }

    public decimal Total
    {
       get { return Cantidad * PrecioUnitario; }
    }
 }
```

9. Aquí se define la entidad principal del sistema. La propiedad Total es calculada y no necesita ser ingresada manualmente.

En el archivo IVentaRepository.cs copiar el siguiente código

```
public interface IVentaRepository
 {
```

```
    Task<List<Venta>> ObtenerVentasAsync();
}
```

La interfaz define el contrato. No dice cómo se cargarán los datos, solo qué operación estará disponible.

10. Ahora en el archivo VentaRepository.cs copiar lo siguiente.

```
public class VentaRepository : IVentaRepository
{
    public async Task<List<Venta>> ObtenerVentasAsync()
    {
        await Task.Delay(2000);


        return new List<Venta>
        {
            new Venta { Id =
1, Producto = "Laptop", Categoria = "Tecnología", Cantidad =
2, PrecioUnitario = 550m, Fecha = DateTime.Now.AddDays(-1)
},
            new Venta { Id =
2, Producto = "Mouse", Categoria = "Tecnología", Cantidad =
5, PrecioUnitario = 20m, Fecha = DateTime.Now.AddDays(-2)
},
```

```
            new Venta { Id = 3, Producto
= "Teclado", Categoria = "Tecnología", Cantidad =
3, PrecioUnitario = 35m, Fecha = DateTime.Now.AddDays(-3)
},
            new Venta { Id = 4, Producto
= "Silla", Categoria = "Oficina", Cantidad = 2, PrecioUnitario =
120m, Fecha = DateTime.Now.AddDays(-2) },
            new Venta { Id = 5, Producto
= "Escritorio", Categoria = "Oficina", Cantidad =
1, PrecioUnitario = 250m, Fecha = DateTime.Now.AddDays(-4)
},
            new Venta { Id = 6, Producto
= "Monitor", Categoria = "Tecnología", Cantidad =
4, PrecioUnitario = 180m, Fecha = DateTime.Now.AddDays(-1)
}
        };
    }


 }
```

Task.Delay(2000) simula un acceso a base de datos o archivo.

El método es asíncrono.

El formulario solo consumirá este repositorio, no sabrá cómo se obtienen los datos.

11. Ahora en el archivo VentaService.cs es donde esta el servicio de procesamiento asi que procedemos a crear el siguiente código:

```
public class VentaService
 {
    public async Task<string> ProcesarVentasAsync(List<Venta> ventas)
    {
        return await Task.Run(() =>
        {
            StringBuilder sb = new StringBuilder();


            decimal totalVendido = ventas.AsParallel().Sum(v => v.Total);
            decimal promedio = ventas.AsParallel().Average(v => v.Total);
            Venta ventaMayor = ventas.AsParallel()
                                .OrderByDescending(v => v.Total)
                                .FirstOrDefault();


            var agrupadas = ventas.AsParallel()
                                .GroupBy(v => v.Categoria)
```

```
                .Select(g => new
                {
                    Categoria = g.Key,
                    TotalCategoria = g.Sum(x => x.Total),
                    CantidadVentas = g.Count()
                })
                .ToList();

        sb.AppendLine("===== REPORTE DE VENTAS =====");
        sb.AppendLine($"Total vendido: {totalVendido:C}");
        sb.AppendLine($"Promedio por venta: {promedio:C}");

        if (ventaMayor != null)
        {
            sb.AppendLine($"Venta
mayor: {ventaMayor.Producto} - {ventaMayor.Total:C}");
        }

        sb.AppendLine();
        sb.AppendLine("Ventas por categoría:");
```

```
            foreach (var item in agrupadas)
            {
                sb.AppendLine($"- {item.Categoria}: {item.TotalCateg
oria:C} en {item.CantidadVentas} ventas");
            }

            return sb.ToString();
        });
    }
}
```

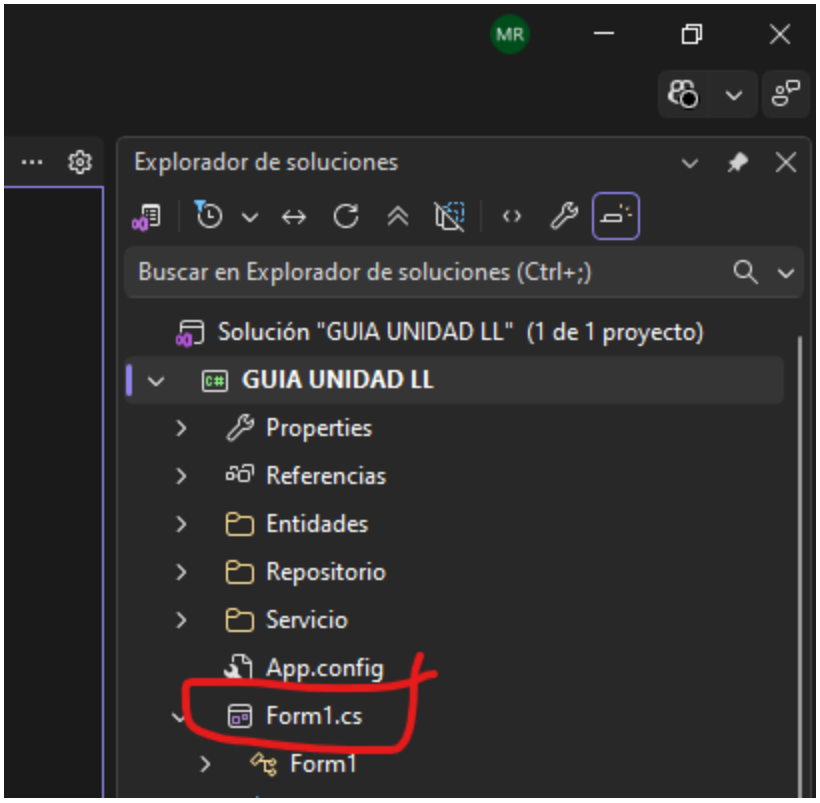
Task.Run(...) ejecuta el procesamiento en una tarea aparte;

Task<string> devuelve un resultado;

AsParallel() aplica procesamiento paralelo a la colección;

se hacen cálculos sin congelar la ventana.

12.Ahora nos dirigimos al Codigo del formulario, Se dirige al explorador de soluciones y se dirige al formulario que tiene de nombre Forms.cs, es darle doble Click y directamente lo mandara al código del formulario.



13. Ahora procedemos a programar el formulario. Nos vamos al código del formulario y copiamos lo siguiente:

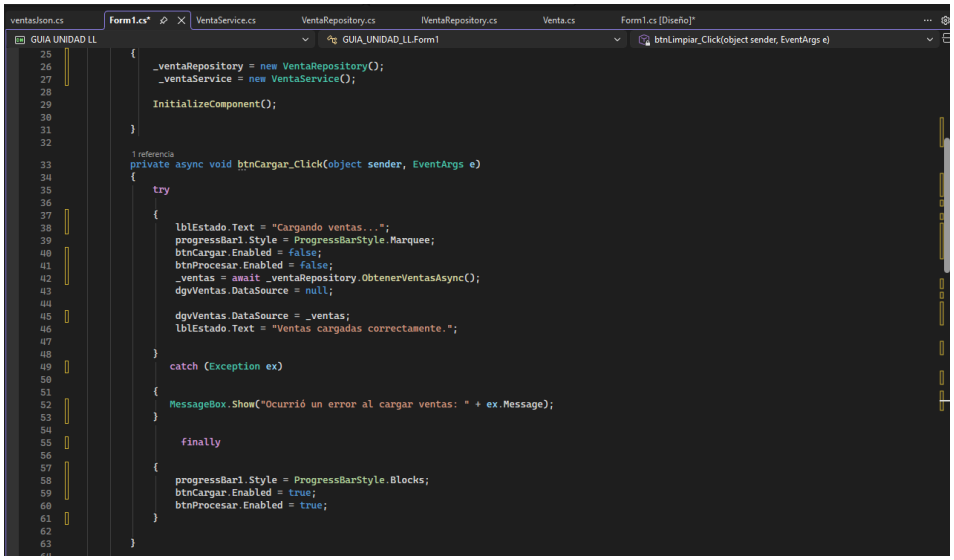
```
public partial class Form1 : Form
{
    private readonly IVentaRepository _ventaRepository;
    private readonly VentaService _ventaService;
    private List<Venta> _ventas;

    public Form1()
```

```
    {
        _ventaRepository = new VentaRepository();
        _ventaService = new VentaService();
        InitializeComponent();
    }
```

14. Ahora generamos un evento al botón Cargar y lo convertimos a un método asíncrono:

```
private async void btnCargar_Click(object sender, EventArgs e)
{
    try
    {
        lblEstado.Text = "Cargando ventas...";
        progressBar1.Style = ProgressBarStyle.Marquee;
        btnCargar.Enabled = false;
        btnProcesar.Enabled = false;

        _ventas = await _ventaRepository.ObtenerVentasAsync();

        dgvVentas.DataSource = null;
```

```
        dgvVentas.DataSource = _ventas;


        lblEstado.Text = "Ventas cargadas correctamente.";
    }
    catch (Exception ex)
    {
        MessageBox.Show("Ocurrió un error al cargar ventas:
" + ex.Message);
    }
    finally
    {
        progressBar1.Style = ProgressBarStyle.Blocks;
        btnCargar.Enabled = true;
        btnProcesar.Enabled = true;
    }
}
```



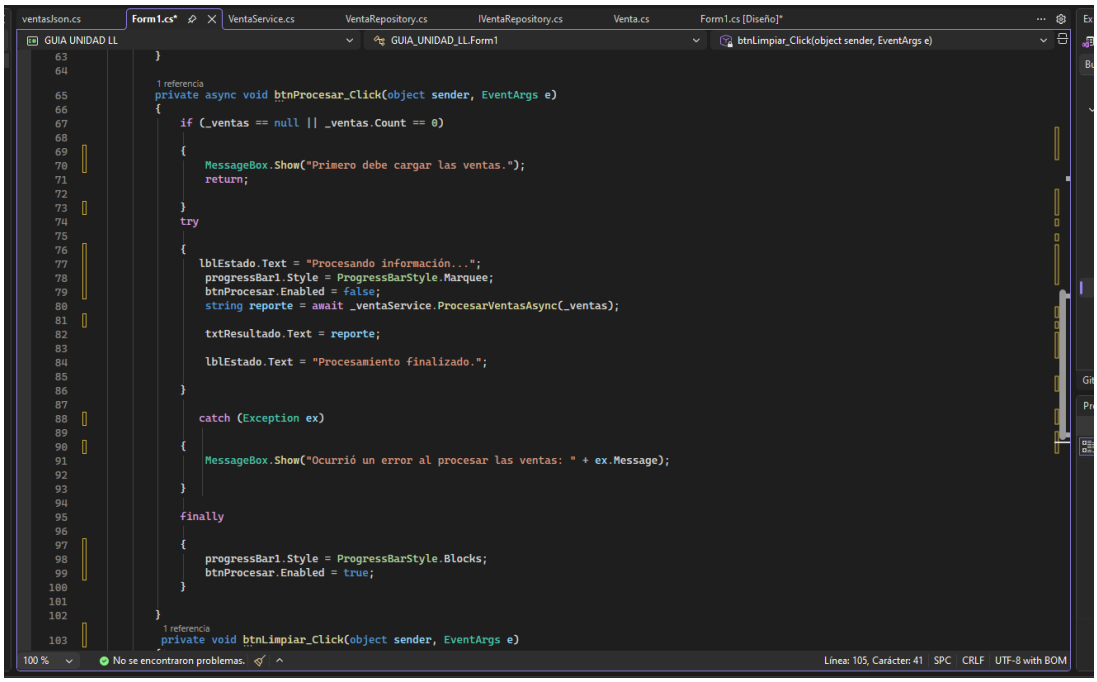
15. se hace lo mismo con el botón de procesar y copiamos lo siguiente:

```
private async void btnProcesar_Click(object sender, EventArgs
e)

{

    if (_ventas == null || _ventas.Count == 0)

    {

        MessageBox.Show("Primero debe cargar las ventas.");

        return;

    }
```

```
try

{

    lblEstado.Text = "Procesando información...";

    progressBar1.Style = ProgressBarStyle.Marquee;

    btnProcesar.Enabled = false;




    string reporte = await
_ventaService.ProcesarVentasAsync(_ventas);




    txtResultado.Text = reporte;
```

```
        lblEstado.Text = "Procesamiento finalizado.";

    }

    catch (Exception ex)

    {

        MessageBox.Show("Ocurrió un error al procesar las
ventas: " + ex.Message);

    }

    finally

    {

        progressBar1.Style = ProgressBarStyle.Blocks;

        btnProcesar.Enabled = true;
```

```
    }

}
```



16. Lo mismo se hace con el botón limpiar

copiamos lo siguiente:

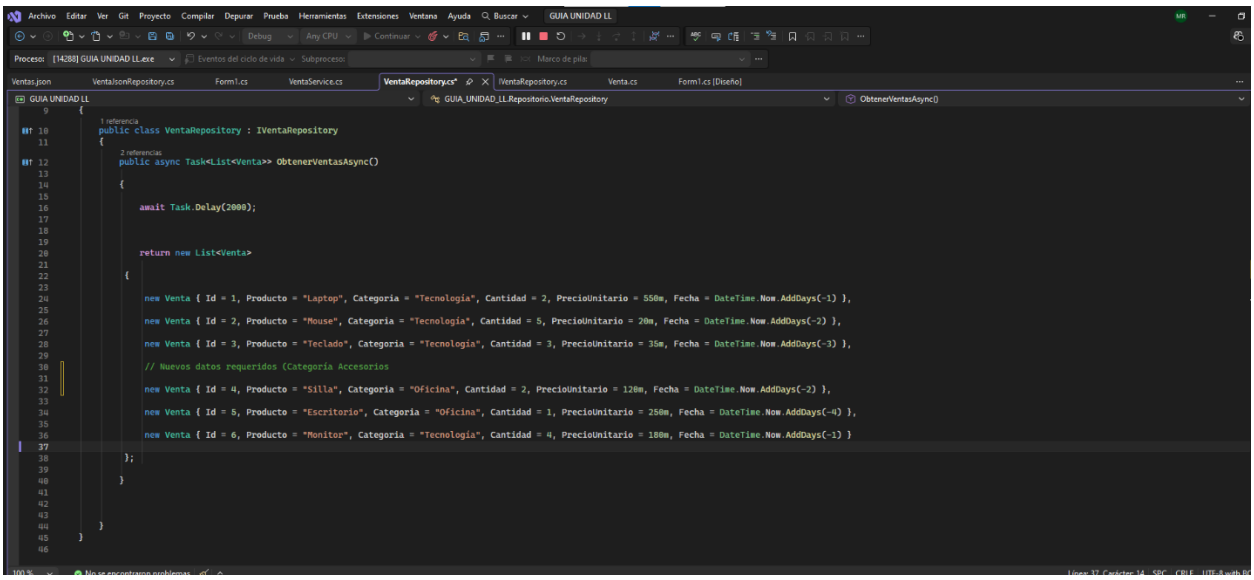
```
 private void btnLimpiar_Click(object sender, EventArgs e)
 {
    dgvVentas.DataSource = null;
```

```
    txtResultado.Clear();
    lblEstado.Text = "Sin datos";
    _ventas.Clear();
}
```

ASIGNACION

17.Agregar una nueva categoría llamada Accesorios y más ventas en el repositorio.

Se agrega la categoría Accesorios y nuevas ventas al repositorio existente

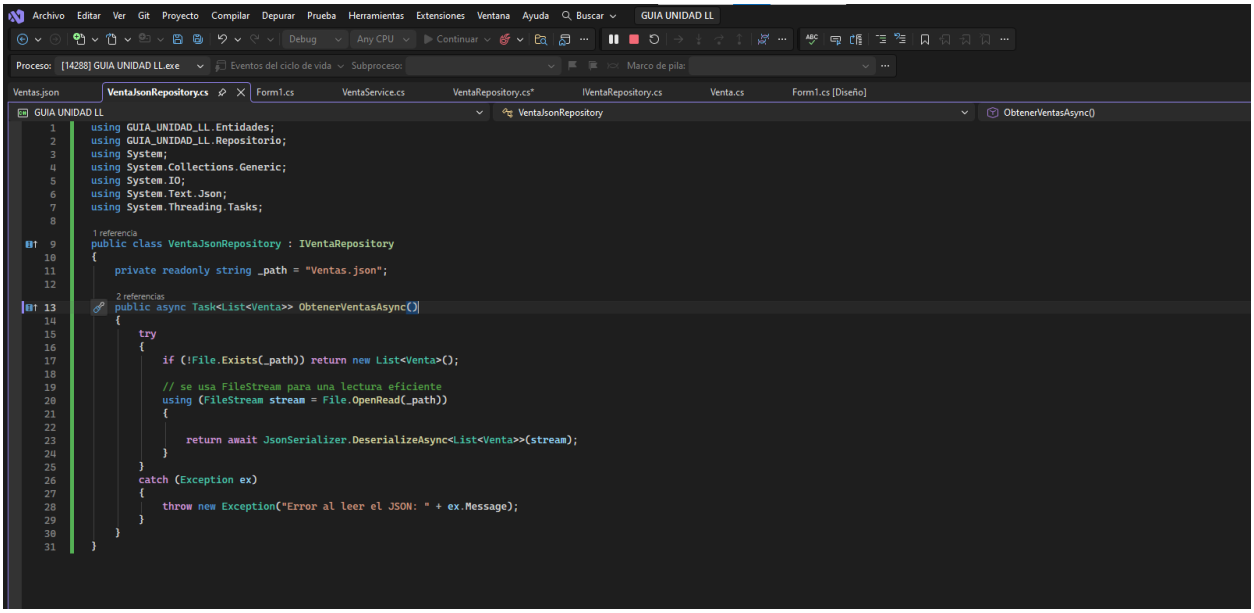


18. Luego en la carpeta repositorio se crea un nuevo elemento que se llame VentasJsonRepository.

Y se pone el siguiente código

```
public class VentaJsonRepository : IVentaRepository
{
    private readonly string _path = "Ventas.json";

    public async Task<List<Venta>> ObtenerVentasAsync()
    {
        try
        {
            if (!File.Exists(_path)) return new List<Venta>();

            // se usa FileStream para una lectura eficiente
            using (FileStream stream = File.OpenRead(_path))
            {

                return await
JsonSerializer.DeserializeAsync<List<Venta>>(stream);
            }
        }
```

```
        catch (Exception ex)
        {
            throw new Exception("Error al leer el JSON: " + ex.Message);
        }
    }
}
```



19. Luego nos vamos a la clase VentaService y Se agregan los nuevos cálculos: cantidad total de productos y fecha de la venta más alta utilizando PLINQ para procesamiento paralelo

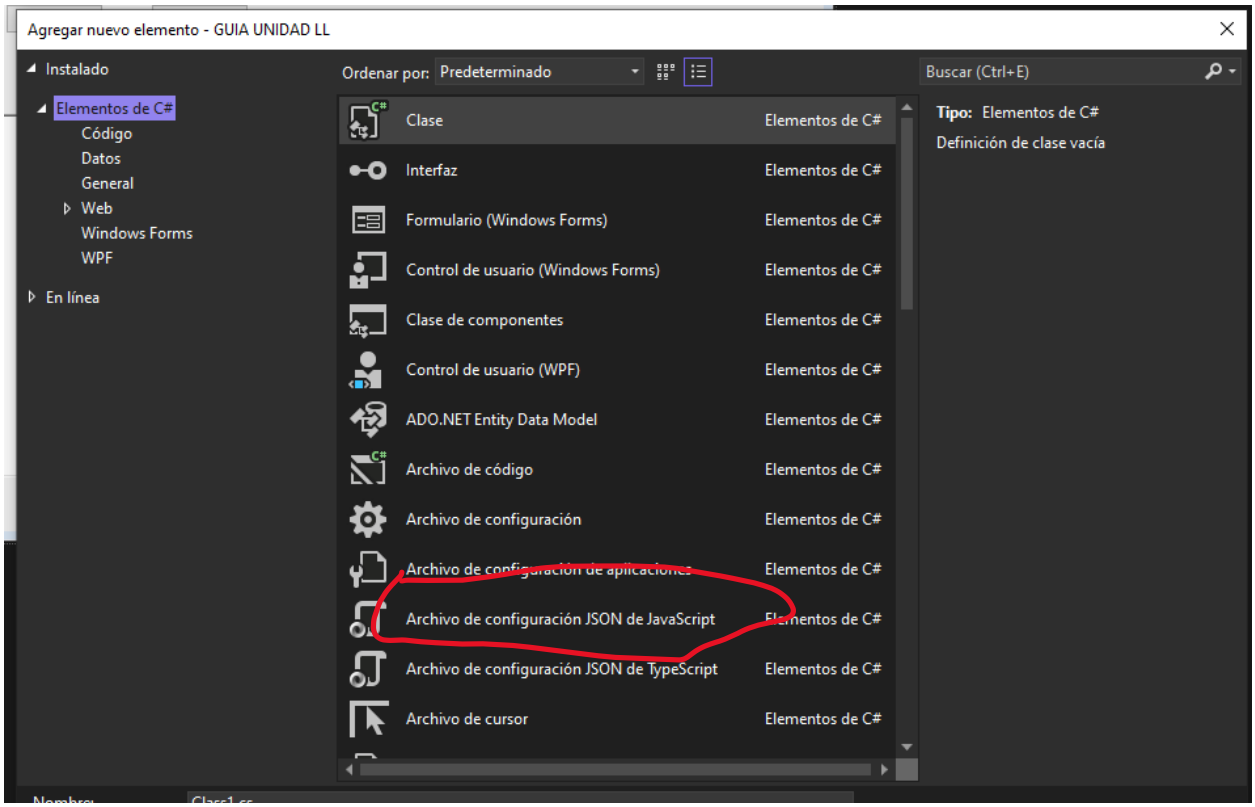
Se agrega el siguiente código.

```
    public async Task<string>
ProcesarVentasAsync(List<Venta> ventas)

    {

        return await Task.Run(() =>

        {

            StringBuilder sb = new StringBuilder();

            decimal totalDinero = ventas.AsParallel().Sum(v =>
v.Total);

            // NUEVOS REQUERIMIENTOS DE LA IMAGEN
            int totalProductos = ventas.AsParallel().Sum(v =>
v.Cantidad); // Cantidad total

            Venta ventaMayor = ventas.AsParallel()
                .OrderByDescending(v => v.Total)
```

```
                .FirstOrDefault(); // Obtiene el objeto con mayor
venta [cite: 130-132]


            sb.AppendLine("===== REPORTE ACTUALIZADO
=====");
            sb.AppendLine($"Total vendido: {totalDinero:C}");
            sb.AppendLine($"Cantidad total de productos:
{totalProductos}"); // Requerimiento 1
        if (ventaMayor != null)
            {
                sb.AppendLine($"Venta más alta:
{ventaMayor.Producto} ({ventaMayor.Total:C})");
                sb.AppendLine($"Fecha de venta más alta:
{ventaMayor.Fecha:dd/MM/yyyy HH:mm}"); //
Requerimiento 2
            }
              return sb.ToString();


         });
      }
   }
}
```

20. Luego te vas a solución a explorador de soluciones y le das click derecho, Luego le das a nuevo elemento y Creas un archivo de configuración JSon y lo llamas Ventas.json (Visual estudio tiene dos configuraciones Elige Archivo JSON (JavaScript).

- ***Razón:*** **JSON significa *JavaScript Object Notation*, por lo que es la opción más "natural". TypeScript es solo una versión extendida de JavaScript, pero el archivo final será el mismo.**



21. Luego pones este código que son los productos de la lista para que te aparezcan en tu datagrieview

[

{

"Id": 1,

"Producto": "Laptop",

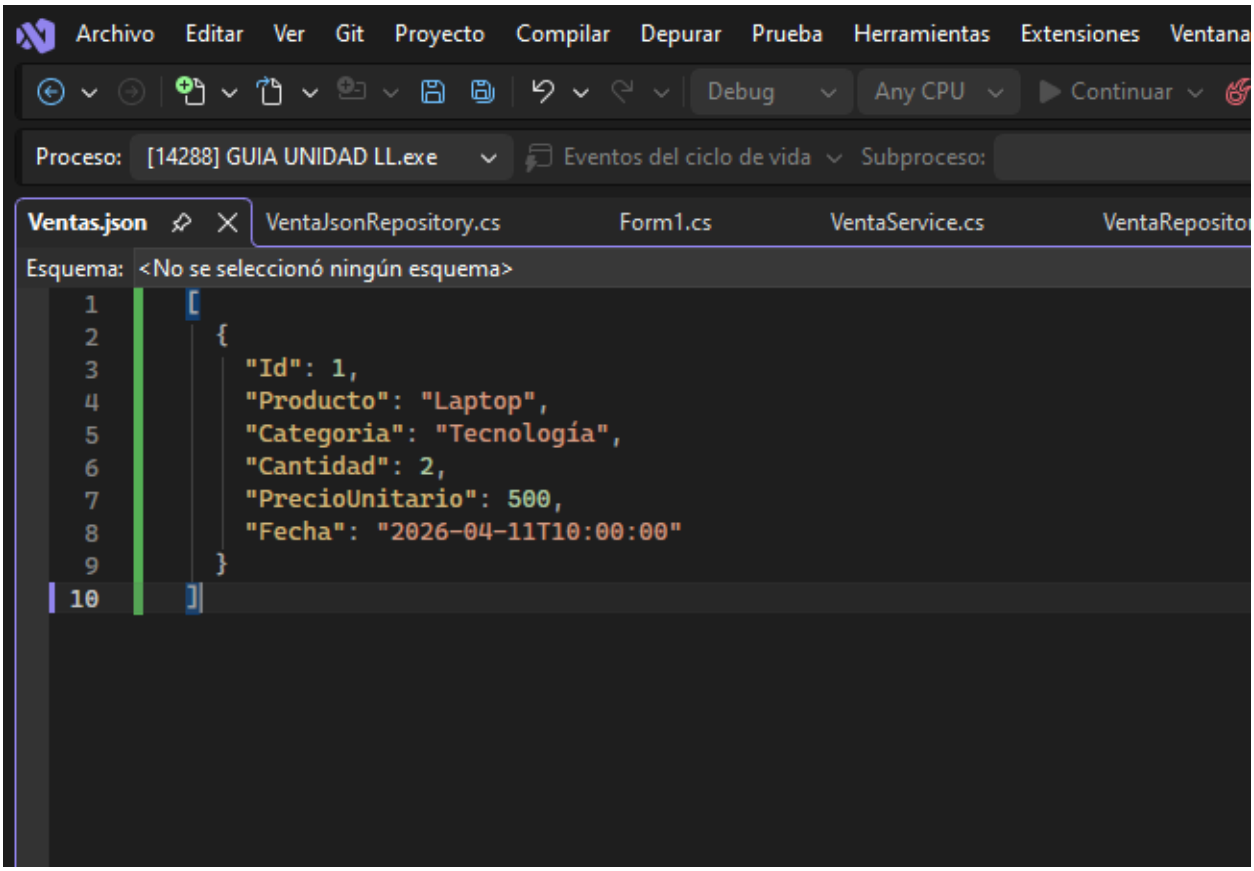
"Categoria": "Tecnología",

"Cantidad": 2, "PrecioUnitario": 500,

"Fecha": "2026-04-11T10:00:00"

}

]



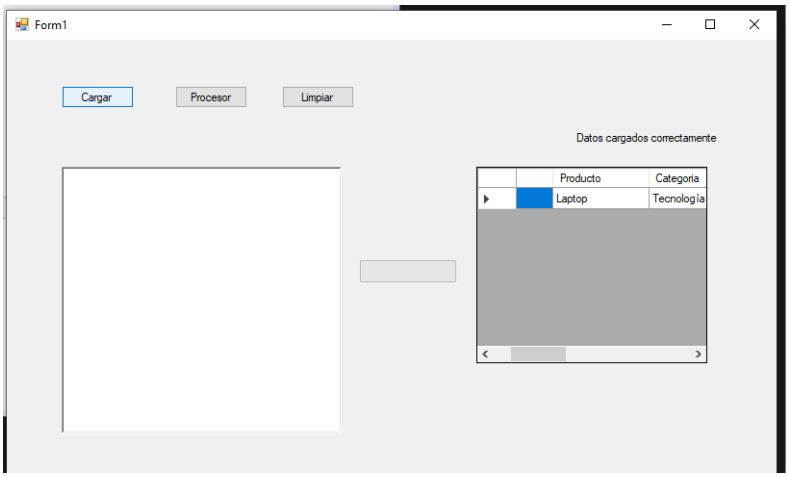
22. Busca la parte del código de tu formulario que se llama igual que la clase (public Form1()). Allí es donde se decide qué repositorio se va a usar. Debes sustituir la línea donde llamabas a VentaRepository por la nueva de VentaJsonRepository

Que quede tal que si:

```
3 referencias
public partial class Form1 : Form

{

    private readonly IVentaRepository _ventaRepository;
    private readonly VentaService _ventaService;
    private List<Venta> _ventas;
      1 referencia
     public Form1()

    {
        _ventaRepository = new VentaRepository();
         _ventaService = new VentaService();
        _ventaRepository = new VentaJsonRepository();
        InitializeComponent();

    }
```

23. Luego se ejecuta el Programa y seleccionas el botón Cargar

Y te saldrá la lista de tus productos para tu tienda



24. Si quieres agregar mas productos te vas al código Ventas.json y agregas los demás productos.

{ "Id": 2,

"Producto": "Mouse",

"Categoria": "Tecnología",

"Cantidad": 5, "PrecioUnitario": 20,

"Fecha": "2026-04-11T08:00:00"

},

{ "Id": 3,

"Producto": "Silla",

"Categoria": "Oficina",

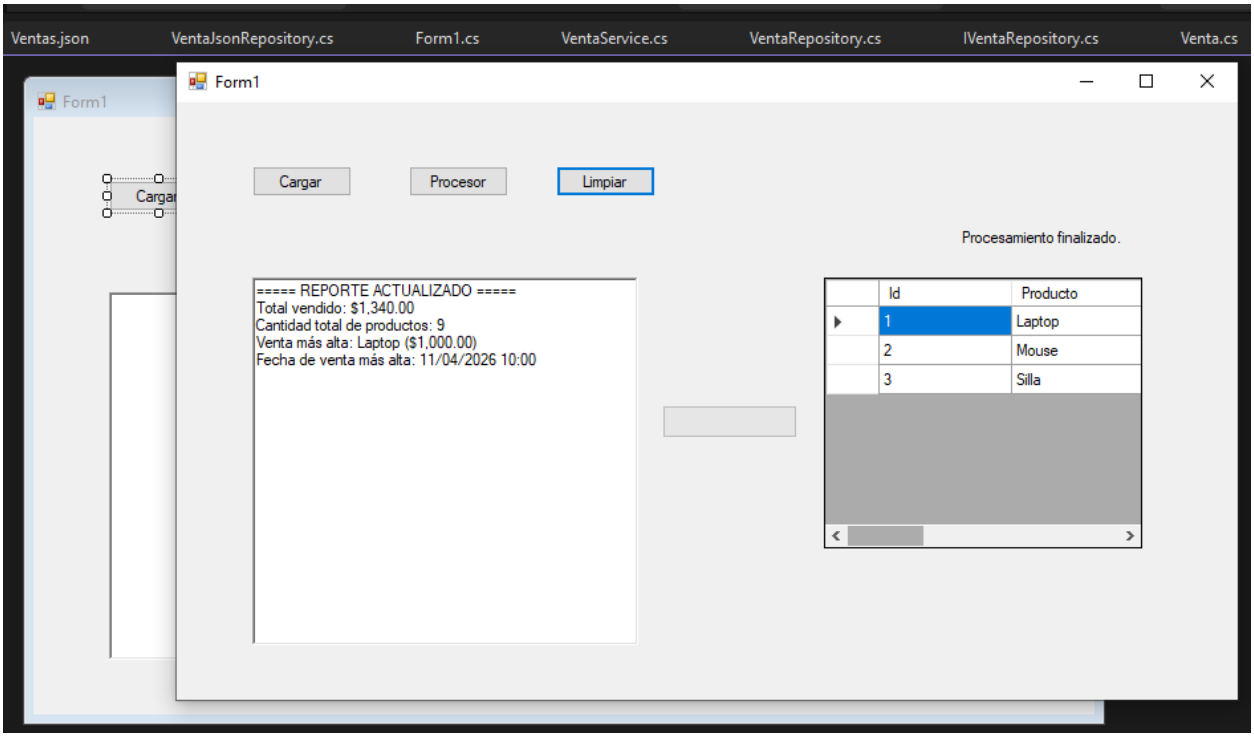
"Cantidad": 2,

"PrecioUnitario": 120, "Fecha":

"2026-04-11T09:00:00"

},

Esquema: <No se seleccionó ningún esquema>

```
[
  {
    "Id": 1,
    "Producto": "Laptop",
    "Categoria": "Tecnología",
    "Cantidad": 2,
    "PrecioUnitario": 500,
    "Fecha": "2026-04-11T10:00:00"
  },
  {
    "Id": 2,
    "Producto": "Mouse",
    "Categoria": "Tecnología",
    "Cantidad": 5,
    "PrecioUnitario": 20,
    "Fecha": "2026-04-11T08:00:00"
  },
  {
    "Id": 3,
    "Producto": "Silla",
    "Categoria": "Oficina",
    "Cantidad": 2,
    "PrecioUnitario": 120,
    "Fecha": "2026-04-11T09:00:00"
  }
]
```

25. como podemos ver se agregaron mas productos ala lista y al darle procesar sale el total vendido, la cantidad total de productos, la venta más alta, y la fecha de la venta más alta